TCP CLIENT–SERVER ECHO PROGRAM

Linux / Ubuntu Socket Programming Lab

Objective

Build a TCP server that receives data from a client and sends the same data back. The client prints the echoed response.

Requirements: Ubuntu/Linux, GCC compiler and two terminal windows. Both programs run on the same machine using 127.0.0.1 and TCP port 8080.

Communication flow

TCP CLIENT — sends “Hello Server”
↓ TCP connection to port 8080 ↓
TCP SERVER — receives and echoes data
↑ Echo response ↑
TCP CLIENT — displays “Hello Server”

1. TCP Server Program

Create a file named **server.c** and enter this code:

```
// server.c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main(void)
{
    int server_fd, client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE];
    ssize_t n;

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) { perror("socket"); return 1; }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&server_addr,
             sizeof(server_addr)) < 0) {
        perror("bind"); close(server_fd); return 1;
    }

    if (listen(server_fd, 5) < 0) {
        perror("listen"); close(server_fd); return 1;
    }

    printf("TCP Echo Server waiting on port %d...\n", PORT);

    client_fd = accept(server_fd, NULL, NULL);
    if (client_fd < 0) {
        perror("accept"); close(server_fd); return 1;
    }

    printf("Client connected.\n");

    while ((n = recv(client_fd, buffer, sizeof(buffer), 0)) > 0) {
        printf("Received: %.*s", (int)n, buffer);

        // send() may send fewer bytes than requested
        ssize_t sent_total = 0;
```

```
        while (sent_total < n) {
            ssize_t sent = send(client_fd, buffer + sent_total,
                                (size_t)(n - sent_total), 0);
            if (sent <= 0) {
                perror("send");
                close(client_fd); close(server_fd); return 1;
            }
            sent_total += sent;
        }
    }

    if (n < 0) perror("recv");
    printf("Client disconnected.\n");

    close(client_fd);
    close(server_fd);
    return 0;
}
```

2. TCP Client Program

Create a file named **client.c** and enter this code:

```
// client.c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main(void)
{
    int sock_fd;
    struct sockaddr_in server_addr;
    char message[BUFFER_SIZE], buffer[BUFFER_SIZE];
    ssize_t n;

    sock_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (sock_fd < 0) { perror("socket"); return 1; }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    if (connect(sock_fd, (struct sockaddr *)&server_addr,
                sizeof(server_addr)) < 0) {
        perror("connect"); close(sock_fd); return 1;
    }

    printf("Connected to TCP Echo Server.\n");
    printf("Enter a message: ");

    if (fgets(message, sizeof(message), stdin) == NULL) {
        close(sock_fd); return 1;
    }

    size_t len = strlen(message), sent_total = 0;
    while (sent_total < len) {
        ssize_t sent = send(sock_fd, message + sent_total,
                            len - sent_total, 0);
        if (sent <= 0) {
            perror("send"); close(sock_fd); return 1;
        }
        sent_total += (size_t)sent;
    }

    // Tell the server that the client has finished sending
    shutdown(sock_fd, SHUT_WR);

    printf("Echo from server: ");
    while ((n = recv(sock_fd, buffer, sizeof(buffer), 0)) > 0)
        fwrite(buffer, 1, (size_t)n, stdout);

    if (n < 0) perror("recv");
    close(sock_fd);
    return 0;
}
```

3. Compile the Programs

```
gcc server.c -o server
gcc client.c -o client
```

4. Run the Lab

Terminal 1 – Server

```
./server
TCP Echo Server waiting on port 8080...
```

Terminal 2 – Client

```
./client
Connected to TCP Echo Server.
Enter a message: Hello Server
Echo from server: Hello Server
```

Expected server output

```
TCP Echo Server waiting on port 8080...
Client connected.
Received: Hello Server
Client disconnected.
```

5. Important Socket Functions

Function	Purpose
socket()	Creates a TCP socket.
bind()	Assigns an IP address and port to the server socket.
listen()	Marks the server socket as ready for incoming connections.
accept()	Accepts an incoming client connection.
connect()	Connects the client to the server.
send()	Sends data through the socket.
recv()	Receives data through the socket.
shutdown()	Disables one direction of communication.
close()	Closes the socket descriptor.

6. Student Experiments

1. Change the port from 8080 to 9090 in both programs and test again.
2. Change the client message and observe the server output.
3. Send a message containing approximately 100 characters.
4. Start the client before the server and explain the connection error.
5. Run the server and client in separate terminal windows.
6. Modify the server to accept multiple clients sequentially.

7. Lab Questions

1. What is the difference between TCP and UDP?
2. Why does the server call bind() before listen()?
3. What is the purpose of accept()?
4. Why does the client use 127.0.0.1?
5. What is the difference between shutdown() and close()?
6. Why should programs check the return values of send() and recv()?

8. Key Learning Points

TCP connection setup, client-server communication, IP addresses, port numbers, socket creation, data transfer and orderly socket closure.

Note: This is a basic single-client echo server. TCP is a byte stream, not a message-based protocol. The client calls shutdown(sock_fd, SHUT_WR) after sending its message so the server can detect the end of input and finish the echo operation.